LostChain - Objets perdus et retrouvés sur le campus

LostChain permet aux membres d'un campus de déclarer des objets perdus ou retrouvés et d'organiser leur restitution. Le projet propose un prototype autonome et une intégration blockchain locale fonctionnelle.

Deux rôles uniquement : Student et Staff, avec les mêmes permissions. Aucun administrateur applicatif, aucun Admin Dashboard et aucune approbation administrative des utilisateurs ou des déclarations. Le déclarant décide des demandes concernant ses propres objets.

1. État actuel

Partie	État
Interface React et navigation	Implémentées
Prototype avec inscription/connexion simulées et localStorage	Fonctionnel, indépendant de la blockchain
CampusIdentity et LostItemRegistry	Implémentés et testés
MetaMask et Ethers.js sur Hardhat local	Intégrés
API de métadonnées et stockage SQLite	Fonctionnels localement
Restitution avec confirmation des deux participants	Implémentée et testée
Tests automatisés et builds	Dernière vérification : 31 tests frontend, 26 tests contrats, 5 tests d'intégration réussis ; TypeScript et deux builds réussis
Réseau Sepolia dans la configuration Hardhat	Préparé ; variables privées à fournir
Connexion de l'application à Sepolia	À implémenter ; l'application impose actuellement la chaîne locale 31337
Déploiement public des contrats et hébergement	À réaliser
Vérification des emails campus	Non implémentée

Le parcours manuel local a été validé avec plusieurs comptes : inscription, déclaration, demande, acceptation, confirmations, statut Returned conservé après actualisation et impossibilité de réclamer à nouveau l'objet rendu. Cela ne constitue pas un audit de sécurité pour un usage public.

2. Technologies et architecture

- Frontend : React 19, React Router 7, Vite 6, CSS et Lucide React.

- Blockchain : Solidity 0.8.34, Hardhat 3, Ethers.js 6 et MetaMask.

- Serveur actif en mode blockchain : Node.js 24+, API HTTP et SQLite via `node:sqlite`.

- Tests : Node Test Runner pour le frontend et l'intégration ; Mocha/Ethers pour les contrats.

```
Navigateur : pages React et contexte applicatif
  ■■ Mode prototype → repository localStorage
  ■■ Mode blockchain → repository Ethers.js
       ■■ MetaMask → CampusIdentity + LostItemRegistry sur Hardhat
       ■■ /api/chain → API Node.js → SQLite et lectures blockchain
```

Les pages utilisent une interface de données commune. Le choix du fournisseur se fait dans `src/context/AppContext.jsx`. Les données du prototype ne sont pas migrées automatiquement vers les contrats.

3. Installation et démarrage rapide

Prérequis : Node.js **24 ou supérieur**, npm, et Chrome ou Firefox avec MetaMask pour le mode blockchain. Utiliser des identités et comptes de test.

Depuis la racine du dépôt :

```
npm ci
npm --prefix blockchain ci
npm run local:start
```

Ouvrir **http://127.0.0.1:5174** dans le navigateur où MetaMask est installé. Conserver le terminal propriétaire des services ouvert.

`local:start` :

1. Compile les contrats.

2. Démarre le nœud Hardhat si nécessaire.

3. Réutilise un déploiement local valide ou déploie CampusIdentity puis LostItemRegistry.

4. Démarre ou réutilise l'API et le frontend blockchain.

5. Refuse de remplacer un service étranger qui occupe un port.

Si tous les services tournent déjà, la commande l'indique et rend la main : c'est normal. Elle ne lance pas de doublons. Son arrêt ne termine que les services qu'elle a elle-même démarrés.

Ports configurés

Service	Adresse
Prototype autonome	http://127.0.0.1:5173
Frontend blockchain local	http://127.0.0.1:5174
RPC Hardhat	http://127.0.0.1:8545
API de métadonnées	http://127.0.0.1:3002

Le frontend Vite relaie `/api/chain` vers le port 3002. Garder le même hôte (`127.0.0.1`) pour les parcours documentés afin d'éviter les différences d'origine et de session.

4. Configuration MetaMask locale

Installer l'extension et créer le portefeuille

1. Ouvrir Chrome ou Firefox, de préférence avec un profil de navigateur dédié aux tests.

2. Aller sur le site officiel MetaMask et choisir **Get MetaMask**, puis son navigateur.

3. Dans la boutique officielle, installer **MetaMask** : sur Chrome, **Ajouter à Chrome**, puis **Ajouter l'extension**. Utiliser la version standard, pas MetaMask Flask.

4. Épingler l'extension avec l'icône puzzle du navigateur pour retrouver facilement le renard MetaMask.

5. Ouvrir MetaMask et choisir **Créer un nouveau portefeuille**. Pour un portefeuille de développement avec phrase de récupération, sélectionner cette méthode et terminer la création du mot de passe et la sauvegarde de la phrase dans MetaMask.

6. Conserver la phrase de récupération hors ligne et privée. Ne jamais la saisir dans LostChain, GitHub ou le terminal, ni l'envoyer à son binôme. Ne pas utiliser une phrase existante contenant des fonds pour cette démonstration.

7. Ignorer les invitations à acheter ou déposer des fonds : les tests locaux utilisent les ETH fictifs des comptes Hardhat importés ci-dessous.

L'installation de l'extension doit être faite par chaque membre de l'équipe. Ouvrir LostChain dans **le même navigateur et le même profil** ; le navigateur intégré d'un outil de développement peut ne pas fournir MetaMask.

Le portefeuille créé par MetaMask et les comptes Hardhat importés sont distincts : le compte créé automatiquement n'est pas financé par le nœud local. Pour les tests, sélectionner un compte importé avec `chain:accounts`.

Références : installation officielle, création du portefeuille, import d'un compte.

Ajouter et sélectionner le réseau Hardhat

Lancer d'abord `npm run local:start` depuis la racine du projet. Dans MetaMask, ouvrir le sélecteur de réseau sous **Jetons**, puis **Gérer les réseaux** → **Ajouter un réseau personnalisé**. Selon la version, le menu principal propose aussi **Réseaux**. Le menu qui affiche seulement les adresses Ethereum/Bitcoin/Solana du compte n'est pas le sélecteur du réseau actif.

Ajouter un réseau personnalisé :

Champ	Valeur
Nom	LostChain Local
RPC	`http://127.0.0.1:8545`
Chain ID	`31337`
Symbole	`ETH`
Explorateur	Vide

Le bouton **Switch wallet to local network** dans l'application permet aussi de demander ce changement.

Après enregistrement, sélectionner réellement **LostChain Local**. Créer le réseau ne garantit pas qu'il soit actif. Le nom du compte affiché en haut de MetaMask ne détermine pas le réseau : vérifier le libellé réseau séparément.

Importer les trois wallets de test

Pour afficher trois comptes Hardhat financés en ETH locaux :

```
npm run chain:accounts
```

Dans MetaMask : menu des comptes → Ajouter un portefeuille → Importer un compte → coller la clé privée du compte de test choisi. Sélectionner ensuite le compte importé et LostChain Local. Les noms des menus peuvent varier selon la version/langue de MetaMask.

Répéter avec **Local test account 2**, puis **Local test account 3** si nécessaire. Renommer les comptes dans leurs détails, par exemple **Local A - déclarant**, **Local B - demandeur**, **Local C - tiers**. Utiliser les clés affichées sur son propre terminal ; ne pas copier celles d'une capture partagée. Cliquer simplement sur **Ajouter un compte** sous le portefeuille MetaMask crée un compte supplémentaire de ce portefeuille, pas un compte Hardhat importé.

Les comptes Hardhat sont publics : ne jamais leur envoyer de vrais fonds ni les réutiliser sur un réseau public. Les ETH locaux n'ont aucune valeur monétaire. Ne pas publier les sorties de la commande ou les captures contenant des identifiants.

Inscription et connexion

- Ouvrir `/register`, saisir un nom, un email et Student ou Staff.

- Autoriser la connexion du bon compte MetaMask.

- Signer le message d'authentification puis confirmer la transaction d'inscription.

- Un portefeuille déjà inscrit doit utiliser `/login`, pas une nouvelle inscription.

- Un changement de compte ou de réseau invalide l'état utilisateur affiché et nécessite une reconnexion.

- Aucun mot de passe n'est demandé dans ce mode. Le nom et le rôle sont publics sur la chaîne ; l'email reste dans l'API.

Chaque binôme exécute sa propre blockchain locale. Cloner Git ne copie pas les inscriptions, objets, demandes ou bases SQLite d'un autre ordinateur.

Test guidé du wallet et de CampusIdentity

Étape	Action	Résultat attendu
1	Sélectionner Local A et LostChain Local dans MetaMask	Le bon compte est sélectionné ; le réseau est 31337
2	Ouvrir `http://127.0.0.1:5174/register`, remplir un nom/email de test et choisir Student	Formulaire valide, aucune demande de mot de passe LostChain
3	Cliquer sur Connect wallet & register et autoriser Local A	MetaMask connecte cette adresse au site
4	Lire et signer le message LostChain local sign-in	Origine `http://127.0.0.1:5174` et Chain ID 31337 dans le message ; la signature n'est pas une transaction
5	Confirmer la transaction d'inscription sur LostChain Local	Transaction confirmée ; membre inscrit et profil accessible
6	Actualiser la page, puis se déconnecter et se reconnecter	L'inscription reste présente ; la connexion demande une signature si nécessaire
7	Essayer de réinscrire le même wallet	Inscription refusée avec indication de se connecter
8	Modifier le nom dans Profile et confirmer la transaction	Le nom est mis à jour après confirmation et actualisation ; le rôle reste en lecture seule
9	Sélectionner Local B, autoriser ce compte pour le site et l'inscrire avec un autre email comme Staff	Profil distinct, mêmes permissions que Student
10	Passer de A à B et inversement	L'application ne conserve pas les droits ou données personnelles du compte précédent
11	Annuler volontairement une demande MetaMask	Message d'annulation visible ; aucune fausse confirmation de réussite

MetaMask peut ouvrir plusieurs fenêtres successives : **connexion**, **signature**, puis **transaction**. Lire et confirmer chacune. Les frais affichés en dollars restent une estimation visuelle : sur LostChain Local, seuls des ETH de test sont utilisés.

Si le compte sélectionné dans MetaMask affiche **Non connecté** pour le site, cliquer sur **Connecter** et autoriser ce compte. Vérifier à nouveau l'adresse utilisée avant l'inscription : sélectionner un compte dans l'extension ne signifie pas toujours qu'il est autorisé pour l'application.

Pour vérifier une mauvaise chaîne, sélectionner un autre réseau puis tenter une action : LostChain doit demander le réseau local, sans envoyer la transaction. Revenir ensuite sur LostChain Local. Ne pas réaliser ce test pendant une transaction en attente.

Test multi-wallet de restitution

1. Avec **Local A**, déclarer un objet Found et confirmer la transaction. Vérifier sa présence dans Explore après actualisation.

2. Avec **Local B**, se connecter et envoyer une demande avec une explication et, éventuellement, une photo de preuve.

3. Avec **Local A**, ouvrir Received Claims, lire la preuve et accepter la demande après confirmation.

4. Avec **Local B**, confirmer la réception. À ce stade, la demande reste Accepted et l'objet n'est pas Returned.

5. Avec **Local A**, confirmer la remise : la demande devient Completed et l'objet Returned.

6. Actualiser et reconnecter chaque compte : vérifier la persistance et l'interdiction d'une nouvelle demande pour cet objet.

7. Sur un autre objet, utiliser **Local C** pour vérifier qu'un tiers ne peut ni accepter la demande, ni confirmer à la place des participants. Vérifier aussi qu'un membre non inscrit ne peut pas envoyer de demande.

Pour la suite Sepolia, créer un compte personnel de test distinct des comptes Hardhat publics. La configuration locale ci-dessus ne demande aucun faucet. Le réseau public et son financement sont traités dans la section 15 ; l'application n'y est pas encore connectée.

5. Prototype autonome sans blockchain

```
npm run dev
```

Ouvrir l'adresse Vite, normalement **http://127.0.0.1:5173**. Ce mode ne nécessite ni Hardhat, ni MetaMask, ni API.

- Inscription avec nom, email, mot de passe et rôle ; connexion simulée.

- Données sous la clé localStorage `lostchain-prototype-v1`.

- Persistance après actualisation et synchronisation des onglets de la même origine.

- Aucun catalogue d'exemple fixe ni raccourci de compte de démonstration : les listes et la communauté viennent des données créées.

- Les migrations retirent les anciens exemples inutilisés tout en conservant les comptes ayant un historique.

- Les vérificateurs de mot de passe utilisent PBKDF2 et un sel aléatoire, mais le stockage reste inspectable et modifiable dans le navigateur. Ne pas utiliser de vrai mot de passe ou de données sensibles.

Les deux modes ont des comptes et données distincts. Supprimer uniquement la clé du prototype permet de réinitialiser ce mode ; cela ne réinitialise pas Hardhat.

6. Pages et fonctionnalités

Page	Route	Fonction
Accueil	`/`	Présentation, compteurs et communauté dynamiques
Inscription	`/register`	Création du membre selon le mode
Connexion	`/login`	Connexion simulée ou signature MetaMask
Dashboard	`/dashboard`	Déclarations personnelles, demandes et activités
Objet perdu	`/report/lost`	Déclaration d'une perte
Objet retrouvé	`/report/found`	Déclaration d'un objet retrouvé
Explorer	`/explore`	Recherche, filtres et nouveaux signalements
Détails	`/items/:id`	Description, déclarant et actions autorisées
Mes déclarations	`/my-reports`	Rapports du membre
Mes demandes	`/claims`	Demandes envoyées et confirmations
Demandes reçues	`/received-claims`	Décisions du déclarant et restitution
Profil	`/profile`	Informations du membre et état d'inscription

`/auth` et `/connect-wallet` redirigent vers `/login`. Les pages personnelles demandent une connexion ; les annonces publiques peuvent être consultées sans être membre.

Fonctions transversales :

- Recherche par texte, dont nom/catégorie, filtres par catégorie, statut, localisation et tri.

- Choix du déclarant : afficher son nom ou rester anonyme dans l'interface.

- Identifiants internes masqués dans le contenu, sauf identifiant raccourci du déclarant anonyme ; ils restent nécessaires dans les données et URL.

- Photos JPG/PNG/WebP jusqu'à 2 Mo, aperçu et validation.

- Preuve écrite obligatoire pour une demande, photo de preuve facultative.

- Notifications non lues, indicateurs et lecture individuelle ou groupée ; compteur décrémenté et état persisté.

- En prototype, les nouveaux rapports alertent les autres membres déjà présents. En blockchain, les notifications sont reconstruites à partir des événements et actualisées par polling.

- Dialogues avant les actions de décision et de confirmation.

Il ne s'agit pas de notifications push navigateur, SMS ou email.

7. Règles du parcours de récupération

1. Un membre publie un objet **Found**.

2. Un autre membre soumet une demande et ses preuves.

3. Le déclarant reçoit une notification et accepte ou refuse.

4. L'acceptation rejette les demandes concurrentes en attente.

5. Les deux participants peuvent enregistrer un lieu/horaire de rendez-vous.

6. Le demandeur confirme avoir reçu l'objet.

7. Le déclarant confirme l'avoir remis.

8. Après les **deux** confirmations seulement : demande **Completed**, objet **Returned**.

L'ordre des deux confirmations est libre. Une seule personne ne peut pas terminer le parcours ou confirmer pour l'autre.

Autres règles :

- Une demande ne peut concerner qu'un objet retrouvé, pas une annonce Lost.

- Pas de demande sur son propre objet, ni de doublon pour le même membre/objet.

- Une demande annulée ne peut pas être soumise de nouveau par ce membre pour cet objet.

- Seul le déclarant accepte/refuse ; seul le demandeur annule sa demande Pending.

- Le déclarant peut fermer un objet ouvert ou avec des demandes en attente : les demandes Pending sont annulées.

- Un objet avec une restitution acceptée ou déjà rendu ne peut pas être fermé.

- Un membre tiers ne peut pas agir sur une restitution.

- Student et Staff suivent exactement les mêmes règles.

Statuts

Contrat	Valeurs
Type d'objet	Lost, Found
État d'objet	Open, ClaimPending, ClaimAccepted, Returned, Closed

Contrat	Valeurs
État de demande	Pending, Accepted, Rejected, Cancelled, Completed

Dans les cartes de l'interface, les objets actifs peuvent conserver le badge Lost/Found même si le contrat indique ClaimPending ou ClaimAccepted. Les demandes affichent leur propre état.

8. Responsabilités des contrats

CampusIdentity.sol

Inscription unique par portefeuille, choix Student/Staff, lecture du profil, décompte/liste des membres et modification du nom.

Interface utilisée : `registerUser`, `isRegistered`, `getUserProfile`, `updateUserProfile`, `getUserCount`, `getRegisteredUsers`.

Le contrat actuel ne stocke ni email ni mot de passe ; il ne permet pas de changer le rôle après inscription. Il ne vérifie pas l'appartenance réelle au campus et ne possède pas de parcours d'approbation administrative.

LostItemRegistry.sol

Référence CampusIdentity à la construction ; gère déclarations, demandes, décisions, annulations, fermeture et double confirmation.

Interface utilisée : `reportLostItem`, `reportFoundItem`, `submitClaim`, `acceptClaim`, `rejectClaim`, `cancelClaim`, `confirmHandover`, `closeItem`, `getItemDetails`, `getClaimDetails`, `getItems`, `getClaimsForItem`, `getItemCount`, `getPendingClaimCount`.

Coordination du binôme : comparer signatures, événements, structures et valeurs des enums avant de modifier un contrat. Une modification peut nécessiter une adaptation du repository, du serveur, des tests et un nouveau déploiement. Une ABI générée manuellement ne remplace pas un contrat compatible.

9. Données, confidentialité et persistance

Données	Mode blockchain
Nom, rôle, adresse et inscription	Contrat CampusIdentity, publics
Titre, catégorie, description, localisation et états	Contrat LostItemRegistry, publics
Email	SQLite ; renvoyé au membre propriétaire de la session
Photo d'annonce, date saisie, préférence nom/anonyme	Métadonnées SQLite ; annonce publique
Preuve écrite, photo de preuve et rendez-vous	SQLite ; accès limité aux participants par l'API
Engagement de preuve	Hash salé dans le message de la demande sur la chaîne
Lecture des notifications	SQLite, par membre
Métadonnées à resynchroniser	File locale du navigateur, par déploiement et wallet

L'anonymat est un choix d'affichage : **les adresses et données publiques du contrat restent consultables**. Il ne faut pas mettre de secret dans une description publique. Les photos de preuve ne sont pas publiées dans les cartes ou notifications ; le contrôle API ne constitue pas un chiffrement de la base.

Les photos sont actuellement stockées comme métadonnées, pas sur IPFS ni dans un bucket externe.

Fichiers générés

- `blockchain/deployments/localhost.json` : adresses, ABI, bloc de déploiement et identifiant unique du déploiement.

- `server/data/chain.sqlite` : métadonnées et sessions de l'intégration locale.

- `server/data/lostchain.sqlite` : ancien backend historique, distinct du mode actif.

- `dist/` : dernier build frontend ; les deux builds utilisent ce même dossier et le second remplace le premier.

L'API sépare les métadonnées par identifiant de déploiement. Les anciennes pièces jointes ne sont donc pas réassociées automatiquement aux nouveaux IDs après une réinitialisation.

Hardhat est éphémère : arrêter puis redémarrer son nœud perd les inscriptions, objets et demandes sur la chaîne. Relancer `local:start` pour détecter l'absence de contrats et redéployer. Il faudra réinscrire les comptes ; conserver SQLite ne restaure pas la blockchain.

10. Configuration déjà réalisée et variables

Élément	Configuration actuelle
Mode Vite blockchain	`.env.chain` : `VITE_DATA_MODE=blockchain`
Réseau local	Hardhat localhost, RPC 8545, chaîne 31337
Déploiement local	`blockchain/scripts/deploy-local.ts`, interdit hors 31337
API locale	`server/chain-index.js`, écoute sur `127.0.0.1`
Origine autorisée API	`APP_ORIGIN`, défaut `http://127.0.0.1:5174`
Base active	`CHAIN_DATABASE_PATH`, défaut `server/data/chain.sqlite`
Port API	`CHAIN_API_PORT`, défaut `3002`
Proxy frontend	`vite.config.js`, `/api/chain` vers 3002
Session wallet	Challenge à usage unique, expiration, signature vérifiée, cookie HttpOnly/SameSite
Actualisation blockchain	Polling environ toutes les 5 secondes et après transaction
Réseau Sepolia Hardhat	Chaîne 11155111 ; variables différées `SEPOLIA_RPC_URL` et `SEPOLIA_PRIVATE_KEY`

Les variables serveur sont lues dans l'environnement exporté. Les fichiers `.env.example` et `blockchain/.env.sepolia.example` sont des modèles, **pas des fichiers de secrets automatiquement chargés par les scripts**. L'ancien `PORT`/`DATABASE_PATH` dans `.env.example` concerne le backend historique.

Changer le port API nécessite aussi d'adapter le proxy Vite. Changer l'origine frontend nécessite d'exporter `APP_ORIGIN` avant le lancement de l'API. La configuration réseau locale est encore imposée dans plusieurs modules : ajouter une entrée Sepolia à Hardhat ne suffit pas à rendre le site compatible.

Ne jamais mettre une clé privée ou un RPC contenant une clé API dans une variable `VITE_*` : ces valeurs sont destinées au navigateur.

11. Commandes disponibles

Toutes les commandes suivantes s'exécutent depuis la racine :

Commande	Effet
`npm run dev`	Frontend prototype

Commande	Effet
`npm run dev:all`	Alias de compatibilité du frontend prototype seul
`npm run local:start`	Démarrage coordonné de l'intégration locale
`npm run chain:accounts`	Affiche trois comptes publics de développement
`npm run chain:node`	Nœud Hardhat seul
`npm run chain:deploy`	Nouveau déploiement local ; ne pas relancer sans vouloir changer de contrats
`npm run metadata:dev`	API locale avec surveillance des fichiers
`npm run dev:chain`	Frontend blockchain seul, port 5174 strict
`npm test`	Tests frontend/prototype et détection des services
`npm run chain:typecheck`	Compilation des contrats et vérification TypeScript
`npm run chain:test`	Tests des deux contrats
`npm run test:integration`	Tests repository + API + contrats locaux
`npm run build`	Build du prototype
`npm run build:chain`	Build du mode blockchain
`npm run preview`	Prévisualisation du dernier build
`npm run preview:chain`	Prévisualisation du build blockchain sur 5174

Ne pas lancer deux commandes concurrentes sur le même port. Pour des terminaux séparés : lancer le nœud, déployer une fois, lancer l'API puis `dev:chain`. Le script coordonné est recommandé pour éviter les doublons.

12. Tests et recette manuelle

```
npm test
npm run chain:typecheck
npm run chain:test
# Avec le nœud local actif et son manifeste présent :
npm run test:integration
npm run build
npm run build:chain
```

Les tests d'intégration déploient leurs propres contrats dans un snapshot EVM puis reviennent à l'état précédent. **Ne pas envoyer de transactions manuelles pendant ces tests**, car le retour au snapshot les annulerait également. Ils utilisent une base de métadonnées temporaire.

Couverture : règles d'accès, égalité des rôles, persistance prototype, notifications et lecture, preuves privées et engagement salé, annulation/fermeture, signatures invalides et rejouées, origine étrangère, stockage saturé, refus MetaMask simulé et reprise d'une synchronisation sans transaction doublonnée.

Recette avec deux comptes :

1. Inscrire deux membres sur le réseau local.
2. Depuis A, publier un objet Found et confirmer la transaction.
3. Depuis B, soumettre une demande avec une preuve.
4. Depuis A, accepter et vérifier les notifications.
5. Confirmer d'un côté : la demande doit rester Accepted, sans Returned.

6. Confirmer de l’autre : vérifier Completed et Returned.

7. Actualiser, se reconnecter et vérifier les mêmes états.

8. Vérifier qu’une nouvelle demande est bloquée.

9. Sur d’autres objets, vérifier refus, annulation, fermeture et interdiction des actions d’un tiers.

Les dialogues réels de l’extension MetaMask font partie de la recette manuelle, pas des tests Node automatisés.

13. Reprise, erreurs courantes et sauvegardes

Symptôme	Action
`Missing script: local:start`	Revenir à la racine LostChain, pas dans `blockchain/`
Ports 3002/5174 occupés	Utiliser `local:start` pour réutiliser les bons services ; ne pas tuer un processus au hasard
Site indisponible	Vérifier le terminal propriétaire et relancer `local:start`
Mauvaise chaîne	Sélectionner LostChain Local 31337 et reconnecter le wallet
Bon compte MetaMask mais mauvais utilisateur	Vérifier les comptes autorisés pour le site et signer avec le compte sélectionné
Wallet déjà inscrit	Utiliser Sign in
Solde ETH local insuffisant	Importer un compte Hardhat financé ; ne pas acheter de vrais ETH
Contrats absents après redémarrage Hardhat	Relancer le démarrage coordonné puis refaire les inscriptions
Transaction confirmée, photo non synchronisée	Utiliser Retry data sync ; ne pas recréer immédiatement la transaction
Session expirée	Signer une nouvelle connexion
Ancienne transaction/nonce après remise à zéro	Vérifier l’état du nœud et l’historique du compte de développement dans MetaMask

Les métadonnées en échec d’envoi restent dans une file localStorage du déploiement et du wallet concernés. Le bouton de reprise n’envoie que les métadonnées.

Pour une copie cohérente de SQLite, arrêter proprement l’API avant de copier la base, ou utiliser un outil de sauvegarde SQLite prenant en charge le mode WAL. La copie du seul fichier principal pendant l’écriture peut être incomplète. Une sauvegarde SQLite ne sauvegarde pas l’état du nœud Hardhat.

14. Arborescence utile

```
src/
  components/                Navigation, cartes, dialogues, notifications, photos
  pages/                     Écrans de l’application
  context/AppContext.jsx     Sélection du mode et fournisseur prototype
  context/ChainProvider.jsx  État blockchain, polling, changements wallet/réseau
  services/prototype/        Validation, transitions, migrations, stockage local
  services/chain/repository.js  Adaptateur Ethers.js et appels API
server/
  chain-app.js               API active, sessions wallet, métadonnées et lectures chaîne
  chain-index.js             Entrée de l’API blockchain locale
  app.js, index.js, auth.js…  Ancien backend conservé, non démarré par local:start
scripts/
  start-local.js             Orchestration locale
  service-check.js           Identification des services sur les ports
  local-wallets.js           Comptes publics Hardhat pour les tests
blockchain/
```

```
  contracts/                CampusIdentity.sol et LostItemRegistry.sol
  test/                     Tests contrats
  scripts/deploy-local.ts   Déploiement et génération du manifeste local
  hardhat.config.ts         Réseaux et compilateur
  .env.sepolia.example      Noms des variables privées à renseigner ultérieurement
  deployments/              Manifestes générés, non versionnés
tests/                      Tests frontend, services et intégration
LOCAL_SETUP.md               Guide local détaillé complémentaire
DEPLOYMENT.md                Préparation et limites du déploiement public
```

Certains anciens adaptateurs et fichiers de données restent dans le dépôt pour conserver le travail précédent. Leur présence ne signifie pas qu'ils sont utilisés par les deux modes actifs. Le serveur actif de l'intégration est celui nommé `chain-*`.

15. Configuration et travail restant pour Sepolia

Préparation externe

À faire : Utiliser un compte MetaMask dédié, différent des comptes Hardhat publics.

À faire : Sélectionner Ethereum Sepolia et obtenir des SepoliaETH via un faucet.

À faire : Configurer un accès RPC HTTPS Ethereum Sepolia.

À faire : Renseigner localement `SEPOLIA_RPC_URL` et `SEPOLIA_PRIVATE_KEY` sans les publier.

À faire : Régénérer les clés API visibles dans des captures déjà partagées.

À faire : Choisir un domaine HTTPS et un hébergement Node.js 24 avec disque persistant, ou une architecture équivalente.

À faire : Choisir la stratégie de stockage des photos et de sauvegarde.

Un compte dédié et le réseau Sepolia ont été préparés manuellement pendant la mise en place. La réception des ETH, la configuration RPC finale et leur présence sur un nouveau poste doivent être vérifiées ; ces réglages privés ne sont pas fournis par Git.

Travail dans le code avant publication

À faire : Paramétrer le réseau dans le repository, l'API, le provider et les libellés UI actuellement locaux.

À faire : Créer un script de déploiement Sepolia distinct avec contrôles et manifeste public sans secret RPC.

À faire : Déployer CampusIdentity puis LostItemRegistry et enregistrer ABI/adresses/bloc.

À faire : Configurer l'API publique derrière HTTPS : origine exacte, cookies Secure, limites de requêtes adaptées et sauvegardes.

À faire : Remplacer les scans complets de l'historique par une indexation incrémentale compatible avec les limites du RPC.

À faire : Étudier le coût en gas de l'acceptation qui parcourt les demandes concurrentes ; fixer une stratégie pour de gros volumes.

À faire : Configurer le reverse proxy `/api/chain` et le fallback `index.html` pour les routes React. Le proxy Vite de développement ne suffit pas à l'hébergement public.

À faire : Définir si une vérification réelle des emails/appartenances campus est nécessaire.

À faire : Refaire les tests et la recette multi-wallet sur Sepolia puis sur le domaine hébergé.

Un frontend statique seul ne peut pas héberger l'API Node/SQLite. Aucun contrat public, service IPFS, stockage cloud, domaine ou hébergement de production n'est actuellement fourni par cette préparation.

Voir DEPLOYMENT.md (DEPLOYMENT.md) pour le passage à une démonstration publique.

16. Git et collaboration

Versionner les sources, contrats, tests, scripts, guides, fichiers de configuration sans secret et les deux `package-lock.json`.

Ne pas versionner :

- Clés privées, phrases de récupération, clés API et vrais fichiers `.env`.

- Bases SQLite et fichiers WAL/SHM.

- `node_modules/`, `dist/`, `blockchain/artifacts/`, `cache/`, `types/` et `deployments/` générés.

- Captures d'écran ou journaux contenant des informations sensibles.

Le `.gitignore` couvre les principaux dossiers générés et fichiers d'environnement ; il ne masque pas automatiquement le contenu secret d'une capture ou d'un fichier source. Vérifier `git status` et les fichiers indexés avant chaque push. Ignorer un fichier déjà suivi ne le retire pas de l'historique Git.

Travailler par branches et annoncer les changements de contrats à l'autre membre avant intégration. Après un changement d'interface Solidity : recompiler, vérifier TypeScript, mettre à jour les appels concernés, tester et décider explicitement si un nouveau déploiement est nécessaire. Ne pas écraser les contrats de l'autre sans comparaison.

17. Documentation complémentaire

- Configuration locale et données (LOCAL_SETUP.md)

- Préparation du déploiement public (DEPLOYMENT.md)

- Modèle de variables Sepolia (blockchain/.env.sepolia.example)

- Modèle de variables serveur (.env.example)

Ce README décrit la version locale et sa préparation au déploiement. Les limites et tâches restantes ci-dessus doivent être traitées avant de présenter LostChain comme un service public prêt pour des données réelles.